

Prototype Implementation and Validation of a Hybrid Authorization Model for Mitigating Administrative Misuse Using Password-less Authentication and Fine-Grained Overrides

Jehan Weerasuriya
Independent Researcher
jehankandy@gmail.com

Abstract

Administrative misuse represents a significant security challenge in modern backend and enterprise systems, particularly when administrators are granted broad and static privileges without sufficient oversight or accountability. Conventional role-based access control (RBAC) models often fail to prevent misuse, as high-risk actions become permanently authorized once a role is assigned.

This work presents the design, implementation, and validation of a hybrid authorization model aimed at mitigating administrative misuse through the integration of password-less authentication, role-based access control, policy-based constraints, and fine-grained, time-bound administrative overrides. A working prototype is implemented using Node.js, Express, and MongoDB to demonstrate how destructive administrative actions are denied by default and permitted only through explicit, auditable override mechanisms.

The prototype serves as a proof-of-concept validating the feasibility and enforceability of the proposed model and highlights its applicability within real-world software engineering contexts where security governance and accountability are critical.

Keywords: *Hybrid Authorization, Administrative Misuse, Password-less Authentication, Role-Based Access Control, Policy-Based Access Control, Fine-Grained Overrides, Software Engineering, Backend Security, Audit Logging*

1. Introduction

Administrative privileges are essential for the maintenance and operation of enterprise software systems; however, they also represent one of the most common sources of security breaches and misuse. Numerous incidents have shown that insider threats, misconfigurations, and excessive privileges can lead to severe system compromise, data loss, and compliance violations.

Traditional access control approaches, particularly role-based access control (RBAC), assign permissions statically based on predefined roles. While RBAC simplifies permission management, it often results in over-privileged administrative roles, where high-risk operations such as user deletion or credential resets are permanently authorized. This lack of contextual restriction and accountability increases the likelihood of misuse.

This research addresses the limitations of static authorization models by proposing and validating a hybrid authorization approach that restricts default privileges while allowing controlled, auditable privilege escalation. The primary objective of this prototype is to demonstrate that administrative misuse can be mitigated without sacrificing operational flexibility.

2. System Architecture Overview

The prototype adopts a layered backend architecture that separates authentication, authorization, override evaluation, and audit logging into distinct components.

The major architectural components include:

- A password-less authentication service using one-time passwords (OTP)
- A hybrid authorization layer combining RBAC and policy-based controls
- A fine-grained override mechanism for high-risk administrative actions
- An audit logging subsystem to ensure accountability and non-repudiation

Node.js and Express are used to implement the backend API, while MongoDB serves as the persistent data store for users, overrides, and audit records. This separation of concerns aligns with software engineering best practices and supports extensibility and maintainability.

3. Authentication Model (Password-less Authentication)

The authentication mechanism implemented in the prototype eliminates static passwords and instead relies on a password-less approach using time-bound one-time passwords (OTP).

The authentication workflow consists of the following steps:

1. The administrator submits an email address to request authentication.
2. The system generates a random OTP with a limited validity period.
3. The OTP is verified by the system to authenticate the user.
4. Upon successful verification, a short-lived JSON Web Token (JWT) is issued.

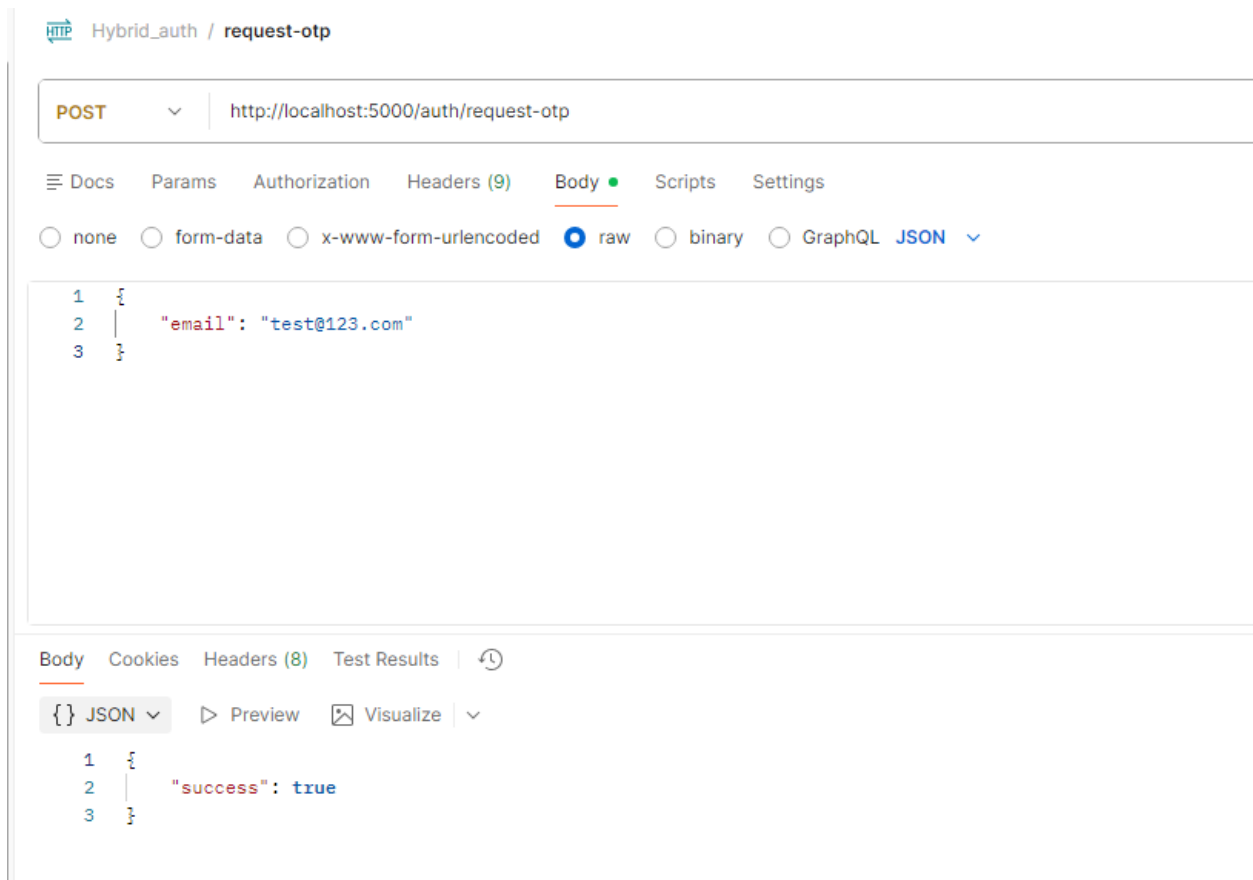


Figure 1: Request OTP for Login to system

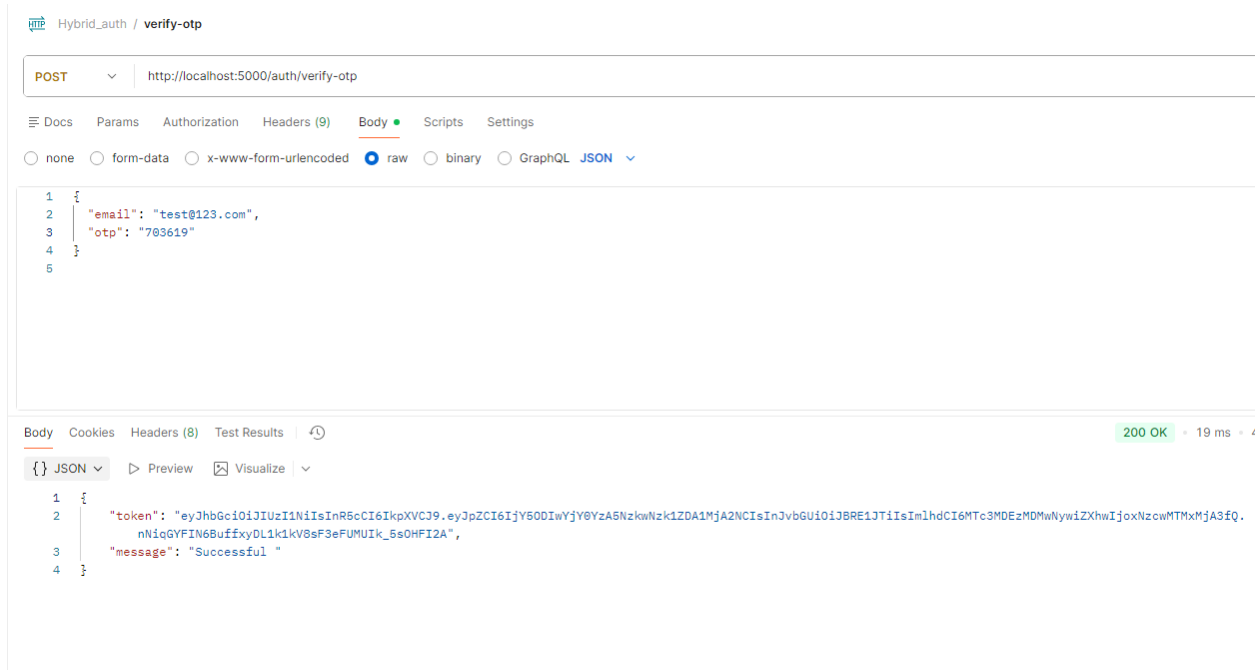


Figure 2: Verify OTP given by system

This approach reduces the risks associated with password reuse, phishing, and credential leakage while simplifying user credential management. The use of short-lived JWTs further limits the impact of token compromise.

4. Hybrid Authorization Model

4.1 Role-Based Access Control (RBAC)

The prototype defines a limited set of base roles, including:

- ADMIN
- SUPERVISOR
- AUDITOR

Each role is intentionally restricted to low-risk or read-only operations by default. Importantly, destructive or sensitive operations are not directly assigned to any role. This design enforces the

principle of least privilege and ensures that role assignment alone does not grant excessive authority.

```
policies > {} policies.json > [] AUDITOR
1  {
2    "ADMIN": [
3      "READ_USER",
4      "UPDATE_USER",
5      "CREATE_USER"
6    ],
7    "SUPERVISOR": [
8      "READ_USER"
9    ],
10   "AUDITOR": [
11     "READ_USER",
12     "READ_AUDIT_LOG"
13   ],
14   "CONDITIONAL": {
15     "DELETE_USER": {
16       "allowed": false,
17       "overrideRequired": true,
18       "constraints": {
19         "justificationRequired": true,
20         "maxDurationMinutes": 60,
21         "dualApproval": false
22       }
23     },
24     "RESET_CREDENTIALS": {
25       "allowed": false,
26       "overrideRequired": true,
27       "constraints": {
28         "justificationRequired": true,
29         "maxDurationMinutes": 30,
30         "dualApproval": true
31       }
32     }
33   }
34 }
```

Figure 3: base role definitions

4.2 Policy-Based and Conditional Authorization

In addition to RBAC, the system incorporates policy-based access control for sensitive operations. High-risk actions, such as deleting user accounts, are classified as conditional actions that cannot be executed without additional authorization checks.

Policies are externalized in a configuration file, allowing authorization rules to be modified without altering application code. This approach enhances flexibility and supports future policy extensions such as contextual or risk-based constraints.

5. Fine-Grained Override Mechanism

To balance security and operational needs, the prototype introduces a fine-grained override mechanism that enables temporary privilege escalation under controlled conditions.

Each override includes:

- The identity of the administrator
- The specific action being overridden
- A justification for the override
- An explicit expiration time

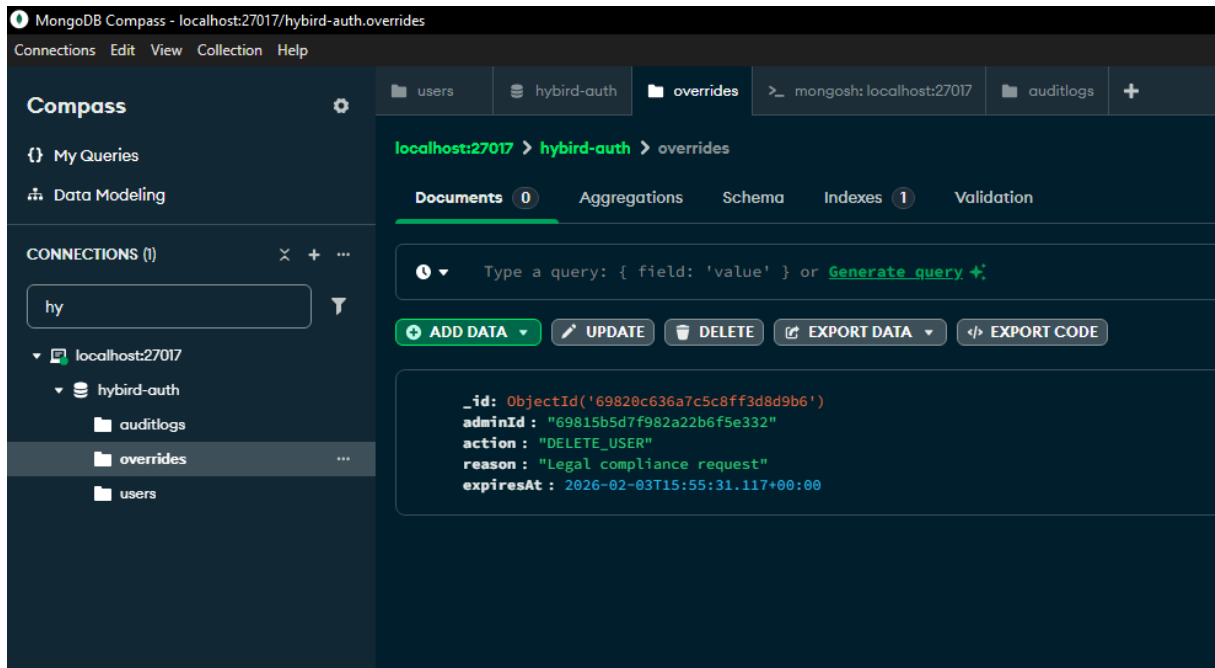


Figure 4: MongoDB override document

Overrides are evaluated at runtime by the authorization middleware. If a valid, unexpired override exists for the requesting administrator and action, the request is permitted; otherwise, it is denied. This ensures that elevated privileges are temporary, intentional, and fully traceable.

6. Authorization Enforcement Flow

Authorization decisions follow a strict evaluation sequence:

1. Authentication verification via JWT
2. Base role permission evaluation
3. Policy-based classification of the requested action
4. Override validation if the action is conditional
5. Final permit or deny decision

This deterministic enforcement flow prevents silent privilege escalation and ensures consistent authorization behavior across all administrative operations.

7. Audit Logging and Accountability

Every privileged operation is recorded in an audit log to ensure accountability and non-repudiation.

Audit records include:

- Administrator identifier
- Action performed
- Whether an override was used
- Timestamp of execution

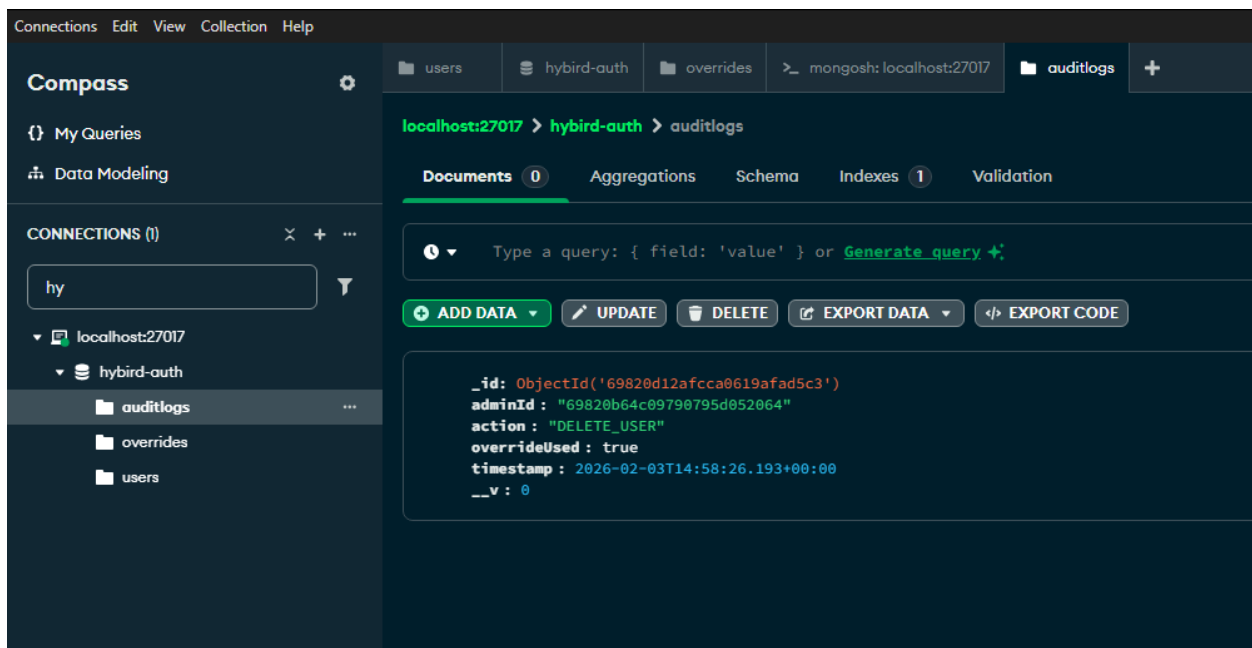


Figure 5: audit log

Audit logs support compliance auditing, forensic investigations, and post-incident analysis, making them a critical component of the overall security architecture.

8. Prototype Validation and Testing

The prototype was validated using Postman to simulate real administrative workflows. Test scenarios confirmed that:

- High-risk actions were denied in the absence of valid overrides
- Overrides enabled actions only within their defined validity periods
- All privileged actions were consistently logged

These tests demonstrate that the prototype correctly enforces hybrid authorization policies and effectively mitigates administrative misuse.

9. Discussion

The implemented prototype demonstrates that static administrative privilege models can be replaced with more secure and accountable alternatives. By separating base authorization from temporary privilege escalation, the system reduces misuse risk while maintaining necessary administrative flexibility.

The use of externalized policies and fine-grained overrides enhances transparency and makes the system adaptable to evolving security requirements. From a software engineering perspective, the design emphasizes modularity, maintainability, and clear separation of concerns.

10. Conclusion

This research presented a prototype implementation and validation of a hybrid authorization model designed to mitigate administrative misuse using password-less authentication and fine-grained overrides. The results demonstrate that the proposed approach is feasible, enforceable, and suitable for real-world backend systems.

Future work may extend the model with dual-approval mechanisms, contextual risk assessment, and alignment with Zero Trust security architectures.

11. Availability

The prototype implementation and supporting documentation are shared as part of the MSc Software Engineering research submission and are publicly available as a technical report on ResearchGate.